

IT461

Practical Machine Learning

Lahjati: Analyzing Arabic Speech Signals for Gulf Dialect Classification

Prepared by

Student	ID	Role
Lamees Alghamdi	444201177	Team leader
Lamya Alnahdi	443202647	Team member
Maha Alruwais	444200749	Team member
Norah Almadhi	444200890	Team member
Maha Albakr	444201108	Team member

Supervised by

Dr. Abeer Aldayel

Table of Contents

1. Introduction.....	4
1.1 Problem and Significance.....	4
1.2 Background Information on the problem domain	4
1.3 Intended Task.....	5
2. Related Works.....	6
3. Dataset	7
3.1 Used datasets	7
3.2 Data preprocessing	9
3.3 Final Preprocessed Dataset.....	9
4. Methods	10
4.1 Machine Learning Algorithms Employed	10
4.2 Justification of Method Selection.....	11
4.3 Feature Engineering Techniques	11
5. Experiment.....	12
5.1 Training Procedure, Feature Engineering, and Dataset Splitting	12
5.2 Data Augmentation and Regularization	12
5.3 Model Architecture Overview	12
5.4 Hyperparameter Tuning.....	13
5.6 Computational Resources.....	14
6. Results and Discussion	15
6.1 Overall Performance Comparison	15
6.2 Generalization to Unseen Data.....	15
6.3 Insights and Observations	16
7. Conclusion	17

7.1 Key Findings	17
7.2 Challenges and Limitations	17
7.3 Future work	17
8. Contributions	18
9. References.....	19

Table of Figures

Figure 1: Overview of the intended task illustrating the system input and output.....	5
Figure 2: Example of Casablanca Dataset.....	8
Figure 3: Example of SADA2022 Dataset	8
Figure 4: Learning-curve Summary.....	16

Table of Tables

Table 1: Examples per Class in the Casablanca Dataset.....	7
Table 2: Examples per Class in the SADA2022 Dataset	7
Table 3: Summary Statistics of Datasets.....	7
Table 4: Example row of Casablanca Dataset.....	8
Table 5: Example row of SADA2022 Dataset	8
Table 6: Example row of Lahjati Final Preprocessed Dataset	9
Table 7: Best Hyperparameters Summary Table	14
Table 8: Performance Comparison Table	15
Table 9: Specific Contributions of Team Members in Writing the Proposal	18

1. Introduction

1.1 Problem and Significance.

The Arabic language encompasses a wide range of dialects, each shaped by the history, culture, and geography of its region [1]. Among these dialects, Gulf Arabic holds a distinctive position due to its characteristic pronunciation, rhythm, and vocabulary, which differ noticeably from other major Arabic dialect groups [2]. These differences are often strong enough that speakers from outside the Gulf may find the dialect unfamiliar or difficult to understand.

This linguistic diversity creates a significant challenge when working with spoken Arabic in any analytical or computational context. Systems that interact with speech whether for linguistic study, classification tasks, or technology development must recognize the unique patterns of Gulf Arabic in order to treat it accurately and avoid confusing it with other dialects. Identifying whether speech is Gulf or non-Gulf therefore becomes an essential step toward understanding regional variation and ensuring that speech-related methods respect the real diversity present across the Arab world.

1.2 Background Information on the problem domain

A dialect is a regional variety of a language distinguished by differences in pronunciation, vocabulary, and intonation [1]. In Arabic, these dialects show noticeable acoustic variations that can be detected through speech analysis. Unlike Modern Standard Arabic, which is formal and uniform, spoken dialects are informal and diverse, making them suitable for classification tasks [3]. In this study, the focus is on distinguishing Gulf Arabic from non-Gulf dialects, a division that reflects one of the most prominent regional distinctions in Arabic speech.

Speech signals refer to the raw audio waveforms generated by human speech. To make these signals usable for machine learning, they must be converted into numerical features that capture important acoustic properties. Common techniques include Mel-Frequency Cepstral Coefficients (MFCCs), spectrogram representations, and pitch-related features all of which describe frequency distribution, energy patterns, and temporal changes within the audio signal [4].

Dialect classification is a supervised learning task in which each audio sample is assigned a dialect label [5]. A classifier is trained to learn the relationship between extracted acoustic features and the target labels, enabling it to predict the dialect of new, unseen speech. Successful classification requires understanding both linguistic variation and audio-signal processing. In this work, feature extraction and preprocessing are implemented using Librosa [6], a widely adopted Python library for analyzing and processing speech signals.

1.3 Intended Task

The intended task is to determine whether an Arabic speech signal belongs to the Gulf or non-Gulf dialect category.

This frames the problem as a binary classification task, focusing on identifying the unique acoustic patterns that distinguish Gulf Arabic from other regional varieties.

- Input: a speech signal containing naturally spoken Arabic.
- Processing: extracting the relevant acoustic features such as MFCCs, spectrogram representations, and pitch-based features.
- Output: a binary prediction indicating whether the given audio sample is Gulf or Non-Gulf.

To evaluate the performance of this classifier, standard metrics such as accuracy, precision, recall, and F1-score are used because they are commonly applied in classification tasks to measure how well the system distinguishes between the two dialect categories [7].

To illustrate this process, we include the following figure:

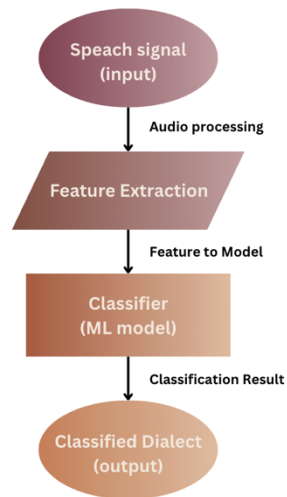


Figure 1: Overview of the intended task illustrating the system input and output

2. Related Works

Arabic Dialect Identification (ADI) has become a crucial research focus in both Automatic Speech Recognition (ASR) and Natural Language Processing (NLP), due to the high variability among dialects and the frequent phenomenon of code-switching between Modern Standard Arabic (MSA) and dialectal Arabic.

Early work by Ali et al. [8] on the Casablanca Dataset [9] examined Arabic broadcast speech with the target task of distinguishing MSA from dialectal Arabic and performing five-dialect classification. Their system used phonetic features, lexical cues, and i-vector representations. The results showed that binary distinctions such as MSA versus dialectal Arabic were nearly separable, achieving 100% accuracy, while fine-grained five-dialect classification was much harder, reaching only about 59% accuracy. This contrast highlights how multi-class ADI is significantly more challenging than binary setups.

The MGB-3 Challenge [10] later introduced a large, multi-genre YouTube dataset to standardize ADI evaluation. The target task was multi-class dialect identification across several Arabic dialects. Systems used a mixture of acoustic features, lexical cues, and deep-learning architectures, including Siamese networks. The best-performing systems achieved 80% accuracy, setting a strong benchmark for multi-class ADI. Although our task is binary, these results indicate typical performance limits for complex ADI setups.

More recent speech studies include Gerazov et al. [11], targeted automatic speech recognition rather than dialect identification, using the SADA dataset [12], a large collection of Saudi and regional dialect audio. Their target task was Transformer-based automatic speech recognition on the SADA dataset, using modern encoder-decoder models, achieving a WER of 40.9 percent. Although not a dialect-ID task, these results highlight how dialectal variation increases model difficulty. SADA remains relevant because it provides modern, multi-dialect speech that informs how Gulf and non-Gulf varieties behave acoustically.

To connect more directly with our binary Gulf vs Non-Gulf task, prior work by Alshutayri and Atwell [3]. evaluated binary dialect identification using the Social Media Arabic Dialect Corpus (SMADC), where dialect labels were assigned at the word level on Twitter data. Their method relied on SVM classifiers with character- and word-based n-gram features, which are particularly effective for written dialects due to distinctive spelling patterns. Their system achieved around 79 percent accuracy, largely because word-level cues are easier to classify compared to acoustic cues in speech. Their findings illustrate that binary text-based dialect classification can outperform multi-class or speech-based systems, although such results are not directly comparable to audio-based ADI tasks like ours.

3. Dataset

3.1 Used datasets

For this project, we use two publicly available Arabic speech datasets that provide labeled dialectal audio suitable for binary Gulf vs Non-Gulf classification.

1. Casablanca [9]:

The Casablanca dataset contains 13,637 labeled Arabic broadcast-speech segments covering 8 dialects shown in Table 1. It includes 6 metadata features such as transcription, gender, dialect label, and code-switching tags. For our task, we map 1500 Emirati examples to the Gulf category and 3000 of remaining dialects (600 each) to Non-Gulf.

Class	Algerian	Egyptian	Emirati	Jordanian	Mauritanian	Moroccan	Palestinian	Yemeni
Num. Rows	2090	1906	1696	1692	1687	1626	1606	1334

Table 1: Examples per Class in the Casablanca Dataset

2. SADA [12]:

The SADA dataset includes approximately 667 hours of Arabic speech from more than 57 Saudi TV shows, forming 4,563 processed rows of labeled segments. It covers 14 dialects, including Saudi (Najdi, Hijazi, Janubi, Shamali) and regional varieties such as Khaliji, Levantine, Iraqi, Egyptian, Yemeni, and Maghrebi shown in Table 2. Each sample includes 48 acoustic and metadata features, making SADA suitable for detailed speech analysis. For our Task, we selected 1500 samples from Saudi and Khaliji dialects for the Gulf class.

Class	Najdi	Hijazi	Unknown	Khaliji	MSA	Egypt	Levantine	NA	Yemen	Shamali	Janubi	Maghrebi	Iraqi	More than 1 speaker
Num. Rows	94611	36170	30867	30320	4302	2172	966	558	407	146	103	41	2	52501

Table 2: Examples per Class in the SADA2022 Dataset

Online Sources:

- **Casablanca:** [Hugging Face link](#)
- **SADA:** [Kaggle link](#)

Summary Statistics of Datasets

Data Set	Number of Examples	Number of Features	Number of Classes
Casablanca	13,637	6	8
Sada2022	4563	48	14

Table 3: Summary Statistics of Datasets

Example of Casablanca Dataset (Algeria Subset):



Figure 2: Example of Casablanca Dataset

Example row of Algeria Subset (showing all columns):

Audio	seg_id	transcription	Gender	Duration
14498_1.wav	14498_1	غا هو خاصك تضور ضويرة بالقهوة تبدل الكراسا و الطباي	M	3.1899

Table 4: Example row of Casablanca Dataset

Example of SADA2022 Dataset:

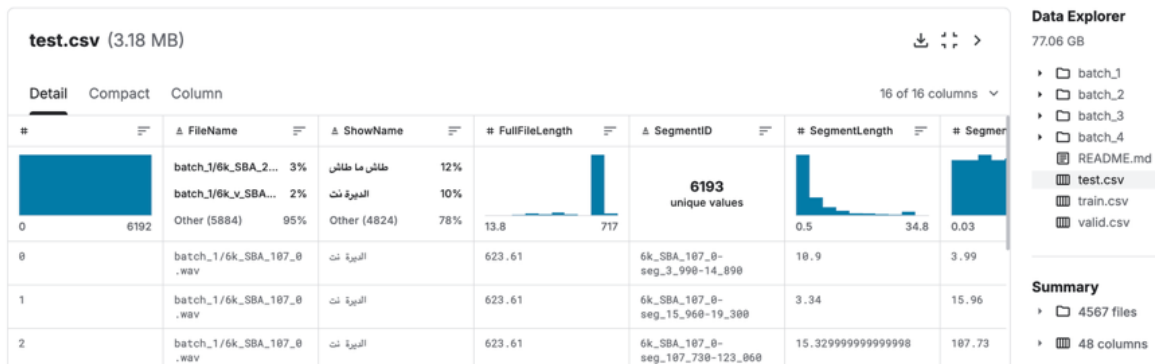


Figure 3: Example of SADA2022 Dataset

Example row (showing all columns):

ID	FileName	ShowName	FullLength	SegmentID
0	batch 1/6k SBA 100 0.wav	الجفاف يقتل الندى	605.3	6k SBA 100 0-seg 249 720-251 840
SegmentLength	SpeakerAge	Speaker Gender	Speaker Dialect	
2.1200000000000045	Elderly	male	Najdi	
Speaker	Environment	GroundTruthText	SegmentStart	
Speaker1	Clean	ووضح كلامك يا مغيث	249.72	
SegmentEnd	ProcessedText	Category		
251.84	ووضح كلامك يا مغيث	درامي		

Table 5: Example row of SADA2022 Dataset

3.2 Data preprocessing

Audio preprocessing was performed to standardize all recordings and improve their quality before feature extraction. These steps ensured clean, consistent audio by removing noise, silence, and volume differences across files.

1. **Resampling:** All audio files were resampled to 16 kHz, converted to mono, and saved in a unified WAV format to ensure consistent sampling characteristics.
2. **Trim Silence:** Silence at the beginning and end of each clip was removed using librosa.effects.trim [6], keeping only active speech content.
3. **Noise Handling:** Background noise was reduced using the noisereduce library [13] to enhance speech clarity and minimize environmental interference. Invalid, corrupted, or extremely short recordings were removed.
4. **Volume Normalization:** Loudness levels were normalized to maintain consistent amplitude across all recordings, preventing volume differences from influencing the model.
5. **Framing & Windowing:** Each recording was divided into short overlapping frames (e.g., 25 ms windows) to prepare the signal for time–frequency feature extraction such as MFCCs.
6. **Dialect Grouping:** Dialects were organized into a binary classification; Gulf (Saudi Arabia, United Arab Emirates), and Non-Gulf (Egypt, Palestine, Algeria, Mauritania, Morocco).

3.3 Final Preprocessed Dataset

The final dataset contains 5999 samples after noise removal, balanced into 3000 Gulf (1) and 2999 Non-Gulf (0). Each entry includes 58 features (55 acoustic features plus filename, duration, and dialect label). Table 6 contains an example row of our final dataset.

Example row (showing all columns):

FileName		duration		Pitch mean		rms mean		spec centroid		spec bandwidth		spec rolloff			
6k_SBA_101_0_seg_0_recon.w av		3.35		148.08		0.034		1335.0		1147.23		2212.08			
f0 std		f0 min		f0 max		f0 range		hnr mean		spectral flatness		spectral entropy		spectral contrast 1	
101.75		50		400		350		4.01		0.01		5.27		15.0	
spectral contrast 2		spectral contrast 3		spectral contrast 4		spectral contrast 5		mfcc 1		mfcc 2					
14.8		19.7		17.5		19.2		-313.01		129.99					
mfcc 3		mfcc 4		mfcc 5		mfcc 6		mfcc 7		mfcc 8		mfcc 9		mfcc 10	
-6.41		41.03		-11.40		-3.21		-8.86		-4.36		2.07		-2.49	
mfcc 11		mfcc 12		mfcc 13		delta 1		delta 2		delta 3		delta 4		delta 5	
-3.39		0.56		2.68		-2.36		-0.54		0.63		0.17		0.17	
delta 6		delta 7		delta 8		delta 9		delta 10		delta 11		delta 12		delta 13	
0.34		0.13		0.19		0.08		-0.27		-0.09		-0.33		-0.34	
delta2 2		delta2 3		delta2 4		delta2 5		delta2 6		delta2 7		delta2 8		delta2 9	
0.02		0.27		-0.24		0.18		0.18		0.096		0.12		-0.13	
delta2 10		delta2 11		delta2 12		dialect									
-0.01		-0.03		1											

Table 6: Example row of Lahjati Final Preprocessed Dataset

4. Methods

This section describes the machine learning methods used in the Lahjati dialect classification system, the justification for selecting these models based on the requirements of the problem, and the feature-engineering techniques applied to convert raw audio signals into meaningful numerical representations.

4.1 Machine Learning Algorithms Employed

To explore different learning paradigms for Arabic dialect classification, we implemented a diverse set of classical and deep learning models. Each model contributes a complementary perspective on how acoustic patterns can be learned and distinguished:

- **Logistic Regression:**
Serves as a simple linear baseline to benchmark model performance.
- **Random Forest:**
An ensemble of decision trees that handles mixed acoustic features effectively and reduces variance through bagging.
- **AdaBoost:**
A boosting method that iteratively focuses on difficult samples, improving robustness against subtle dialect differences.
- **Fully Connected Neural Network:**
Learns complex non-linear relationships between the engineered feature vectors and dialect labels.
- **Recurrent Neural Network (RNN):**
Captures temporal dependencies in speech, such as rhythm, intonation, and prosodic variation.
- **Support Vector Machine (RBF Kernel):**
Well-suited for non-linear decision boundaries and performs strongly with MFCC-based input representations.
- **Convolutional Neural Network (CNN):**
Treats MFCC matrices as time-frequency "images," enabling the model to learn discriminative spectral-temporal patterns.

4.2 Justification of Method Selection

The selected models were chosen to address the main challenges of dialect recognition. Since Arabic dialects exhibit non-linear acoustic differences, methods such as SVM, AdaBoost, and neural networks were included to capture complex decision boundaries. The wide range of extracted features—including MFCCs, spectral measures, pitch, and RMS—benefits from ensemble approaches like Random Forest, which handle mixed feature types effectively. Because speech also contains temporal cues, RNN and CNN architectures were used to model changes in intonation and rhythm across time. Logistic Regression served as a simple baseline to benchmark model performance. Overall, the dataset size was suitable for training both classical machine learning models and deeper networks without severe overfitting, making this combination of methods appropriate for the task.

4.3 Feature Engineering Techniques

Each audio sample was converted into a 58-dimensional feature vector after preprocessing (silence trimming, normalization, noise reduction). The final set includes 55 acoustic features plus file name, duration, and dialect class label. Extracted acoustic features include:

- Pitch-related features: f0_std, f0_min, f0_max, f0_range, hnr_mean
- Spectral features: spectral centroid, spectral bandwidth, spectral rolloff, spectral flatness, spectral entropy
- Spectral contrast (1-5)
- MFCCs (1-13)
- Delta MFCCs (1-13)
- Delta-Delta MFCCs (1-12)

These features capture both spectral and temporal information, forming a compact and informative representation for all trained models. No dimensionality reduction was applied due to the manageable feature size.

5. Experiment

5.1 Training Procedure, Feature Engineering, and Dataset Splitting

The training pipeline ensured that all audio samples were processed consistently for binary classification (Gulf vs Non-Gulf). Each recording was normalized to -23 LUFS with peak limiting and light noise reduction, followed by a pre-emphasis filter to enhance high-frequency cues relevant to dialect distinction. Audio was framed into 25 ms windows with a 10 ms hop and passed through a Hamming window, after which 58 engineered numerical features were extracted: MFCCs, their Delta and Delta² coefficients, RMS energy, spectral centroids, spectral bandwidths, and pitches (F0). These features were compiled into a single dataset (lahjati_dataset.csv) and split using an 80/20 stratified split, preserving the balance between Gulf and Non-Gulf samples. Model validation was performed through stratified 5-fold cross-validation during hyperparameter tuning rather than a separate validation set.

5.2 Data Augmentation and Regularization

No explicit augmentation techniques were used in order to preserve the natural acoustic characteristics of the dialect samples. Generalization was instead improved through model-level regularization. Random Forest used structural controls such as bootstrap sampling, feature sub-sampling, and constraints on depth and minimum split size, while Logistic Regression depended on regularization strength C . AdaBoost controlled learning rate and estimator depth, and SVM used class-weight balancing and RBF kernel parameters. Neural architectures (MLP, RNN, CNN) relied on dropout, activation functions, and learning-rate tuning. Cross-validation across all models provided additional regularization by evaluating performance on multiple folds.

5.3 Model Architecture Overview

The project evaluated both classical and neural architectures. Classical models use parameter-driven structures rather than layers. Random Forest, the strongest performer, consists of many decision trees trained on bootstrapped subsets with randomized feature selection, enabling it to capture non-linear patterns in the MFCC-based and spectral features. AdaBoost builds sequential weak learners whose errors guide the next estimator. SVM uses an RBF kernel to form non-linear decision boundaries. Neural models were also explored: MLPs with two hidden layers, RNNs with recurrent units and dropout, and CNNs with convolutional layers and pooling. While the neural models incorporated deeper representations, their performance was consistently lower than the classical models on this tabular feature set. The ensemble-based Random Forest ultimately learned the feature relationships most effectively.

5.4 Hyperparameter Tuning

This section outlines the hyperparameter tuning process used across the project's machine learning and deep learning models. Most classical models were tuned using **RandomizedSearchCV**, which efficiently samples parameter combinations. The deep learning models were tuned using a manual randomized search, where predefined candidate values were randomly tested based on validation performance rather than full grid evaluation. A summary of best parameters is shown in Table 7.

For **Logistic Regression**, **RandomizedSearchCV** tuned the regularization strength C (0.001 to 100) and solver (lbfgs, saga) over 20 sampled configurations with 5-fold cross validation. The best result used $C \approx 44.58$ and the lbfgs solver.

For **Random Forest**, **RandomizedSearchCV** explored the number of trees (100 to 400), maximum depth (10, 20, 30, None), max features (sqrt), min samples split (2 to 10), min samples leaf (1 to 4), bootstrap (True), and class weight (None or balanced subsample). The best configuration used 352 trees, max depth None, max features sqrt, min samples split 7, min samples leaf 1, bootstrap True, and class weight balanced subsample.

For **AdaBoost**, **RandomizedSearchCV** evaluated the number of estimators (50 to 300), learning rate (0.01 to 1.0), and base tree depth (1 to 5). The best result used 201 estimators, learning rate ≈ 0.61 , and depth 5.

For the **Neural Network MLP**, manual randomized search explored hidden layer sizes ((50), (100), (100, 50)), activation functions (relu, tanh), solvers (adam, sgd), learning rates (0.0001 to 0.01), and max iterations (200 to 600). The best model used relu activation, hidden layers (100, 50), learning rate ≈ 0.00993 , max iterations 330, and the adam solver.

For the **Recurrent Neural Network**, manual randomized search tested units (32, 64, 128), dropout values (0.2 to 0.4), and learning rates (0.001 to 0.0005). The best configuration used 64 units, dropout 0.4, and learning rate 0.0005.

For **Support Vector Machine**, **RandomizedSearchCV** sampled C (0.001 to 1000), gamma (scale, auto, and 0.0001 to 10), rbf kernel, and class weight (None or balanced). Out of 40 configurations, the best parameters were kernel rbf, gamma ≈ 0.00018 , class weight balanced, and $C \approx 7.01$.

For the **Convolutional Neural Network**, a manual randomized search was applied to architectural choices such as filter sizes, number of convolutional layers, dropout rate, and dense layer size. The best architecture was then validated with 5 fold stratified cross validation, retraining the model in each fold. The mean accuracy across folds was approximately 0.661.

Summary Table of Best Hyperparameters

Model	Best Hyperparameters
Logistic Regression	$C \approx 44.58$, solver = lbfgs
Random Forest	n_estimators = 352, max_depth = None, max_features = sqrt, min_samples_split = 7, min_samples_leaf = 1, bootstrap = True, class_weight = balanced_subsample
AdaBoost	n_estimators = 201, learning_rate ≈ 0.611 , estimator_max_depth = 5
Neural Network MLP	activation = relu, hidden_layer_sizes = (100, 50), learning_rate_init ≈ 0.00993 , max_iter = 330, solver = adam
Convolutional Neural Network	units = 64, dropout_rate = 0.4, learning_rate = 0.0005
Support Vector Machine	kernel = rbf, gamma ≈ 0.00018 , class_weight = balanced, $C \approx 7.01$
Recurrent Neural Network	mean cross validation accuracy ≈ 0.661 with the selected architecture

Table 7: Best Hyperparameters Summary Table

5.5 Model Evaluation

Since this project addresses a binary classification problem, the performance of all models was evaluated using metrics suited for distinguishing between binary classes [7], Gulf and Non-Gulf in our case. Each model was assessed on the held-out test set using accuracy, precision, recall, the F1 score, and the confusion matrix. Accuracy provided an overall measure of how correctly the models classified audio clips into Gulf or Non-Gulf. Precision and recall helped determine how well each model captured true Gulf speech without incorrectly labeling Non-Gulf samples, and vice versa. The F1 score offered a balanced indication of this performance by combining precision and recall into a single measure. The confusion matrix visualized the distribution of correct and incorrect predictions for both classes, making it possible to inspect where misclassifications occurred within the Gulf versus Non-Gulf labels.

5.6 Computational Resources

All experiments were executed in Python using Google Colab [14]. The environment ran on the standard Python 3 Google Compute Engine backend [15] with 2.5 GB were actively used during model training and evaluation out of 12.7 GB of RAM available. The Colab session provided a disk capacity of 107.72 GB, with around 38.17 GB occupied and 69.55 GB available during experimentation. The runtime operated on CPU only, as no GPU or TPU accelerators were enabled throughout the project.

Project files were stored in Google Drive [16], which contained roughly 23.47 GB of used storage at the time of development. The main libraries used included scikit-learn, NumPy, Pandas, Matplotlib, Librosa, TensorFlow, Keras, TQDM, SoundFile, TorchAudio. Additional utilities used for audio processing and dataset handling included pyloudnorm, noise_reduce, tqdm, zipfile, soundfile, fsspec, scipy, seaborn, IPython audio display tools, torchaudio, and Google Drive mounting utilities [17]. This computational setup provided sufficient resources for running classical machine learning algorithms as well as the deep learning models used in the project.

6. Results and Discussion

This section presents and interprets the performance of all implemented machine learning models for Gulf vs. Non-Gulf dialect classification. The results include evaluation metrics and learning curves summaries. Together, these analyses allow us to assess each model’s predictive capability, generalization performance and stability.

6.1 Overall Performance Comparison

Table 8 summarizes the key evaluation metrics: Accuracy, F1-macro score, Recall macro and Precision macro cross all models after hyperparameter tuning and cross validation.

Model	Accuracy	Precision macro	Recall macro	F1 macro
AdaBoost	0.766	0.766	0.766	0.766
Random Forest	0.765	0.766	0.765	0.765
Logistic Regression	0.698	0.699	0.698	0.698
Neural Network MLP	0.685	0.702	0.685	0.678
Convolutional Neural Network	0.685	0.711	0.685	0.675
Support Vector Machine	0.650	0.650	0.650	0.650
Recurrent Neural Network	0.618	0.618	0.617	0.617

Table 8: Performance Comparison Table

The performance table clearly shows that ensemble methods specifically **AdaBoost** and **Random Forest** achieved the highest and most balanced accuracy, precision, recall, and F1-macro scores. These results indicate that tree-based ensembles were the most effective at capturing the structure of the engineered audio features. Classical models also outperformed deep learning approaches, with Logistic Regression showing stable and reliable performance despite its simplicity. In contrast, neural models such as MLP, CNN, and RNN demonstrated lower and less consistent results, suggesting that the aggregated feature representation limited their ability to learn deeper temporal or frequency patterns.

6.2 Generalization to Unseen Data

The learning-curve summary in the bar chart shown in figure 4 reveals important differences in how well each model generalizes beyond the training data. Ensemble methods (Random Forest and AdaBoost) show a perfect training accuracy of 1.00, but their validation accuracy remains in the 0.76–0.77 range. This gap indicates overfitting, but the validation performance is still the highest among all models, meaning that despite memorizing the training data, they generalize better than the rest.

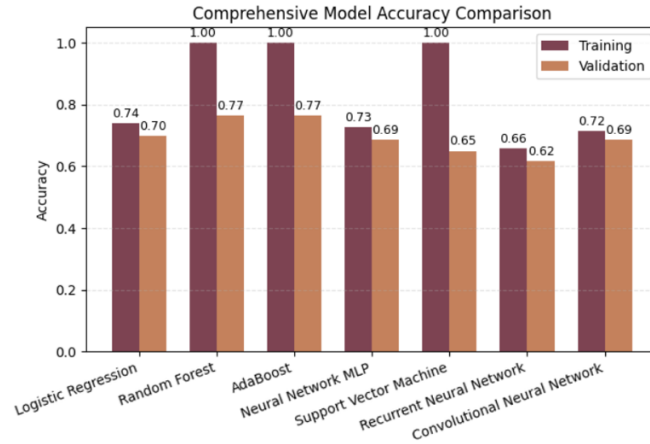


Figure 4: Learning-curve Summary

Logistic Regression displays nearly identical training (0.74) and validation (0.70) accuracy, suggesting **strong generalization with no overfitting**. Its performance is lower than the ensembles, but its stability makes it one of the most reliable models. The MLP and CNN models have moderate training accuracy (0.73 and 0.72) and slightly lower validation accuracy (0.69). Their small gaps indicate that they did not overfit but also did not fully capture the complexity of the data. SVM has a 1.0 training accuracy and validation accuracy (0.65), indicating overfitting. Additionally, the RNN shows close training (0.66) and validation (0.62) performance, suggesting that it also underfits due to the non-sequential nature of the extracted features.

Overall, the results show that Random Forest and AdaBoost generalize the best in terms of actual validation accuracy, despite some overfitting, while Logistic Regression offers the cleanest generalization behavior. Deep learning models and SVM generalize less effectively, either due to underfitting or misalignment between architecture and feature type.

6.3 Insights and Observations

Several key insights emerge from the comparative results across all models. First, the ensemble methods AdaBoost and Random Forest consistently delivered the strongest performance, achieving the highest validation accuracy and the most balanced precision, recall, and F1-scores. Even though both exhibited some overfitting, their validation performance remained high, suggesting that tree-based ensembles are particularly effective for the type of engineered audio features used in this study.

A second observation is that simpler models generalized more cleanly. Logistic Regression showed minimal difference between training and validation accuracy. This suggests that the extracted features included several linearly separable patterns that a simple linear classifier could learn reliably.

In contrast, deep learning models underperformed relative to classical approaches. The MLP, CNN, and RNN showed either underfitting or limited ability to leverage the fixed-length numerical features, which do not preserve the temporal or spectral structure that neural architectures typically depend on. Their lower accuracy and inconsistent validation performance highlight a mismatch between model architecture and input representation.

Finally, the consistent trends across performance metrics, and learning curves highlight a clear pattern: models that rely on strong, feature-based decision boundaries (especially ensembles) perform best, while models that depend on raw sequential structure struggle when the temporal nature of audio is compressed into aggregated features. These findings emphasize the importance of model–feature alignment and suggest that future improvements may come from revisiting the feature extraction pipeline, particularly by incorporating spectrograms or raw-signal deep learning approaches.

7. Conclusion

7.1 Key Findings

The project showed that Arabic dialects can be classified from speech recordings using machine-learning models. Random Forest delivered the strongest overall performance across all evaluation metrics. Logistic Regression demonstrated the best generalization, showing almost no overfitting despite its simplicity. Overall, the engineered acoustic features successfully captured patterns that distinguish Gulf from Non-Gulf speech.

7.2 Challenges and Limitations

Throughout the project, the main difficulty was identifying the most effective features to distinguish between the two dialects. Another limitation was the challenge of achieving a reasonably high accuracy, as improving the model’s performance required multiple training iterations and tuning steps. The similarity between some dialects also made the classification task more demanding, which affected the model’s ability to separate classes clearly.

7.3 Future work

- Extend the system to support multi-class dialect classification beyond the binary setup.
- Conduct additional testing with new and more diverse datasets.
- Explore alternative feature representations to potentially improve accuracy and model stability.

8. Contributions

This following table summarizes the individual contributions and outlines the roles and responsibilities of each team member in preparing the proposal:

Team Member	Task
Lamees Alghamdi	▪ 2.1 Key Concepts and Background Information ▪ 2.2 Related Work: Automatic Dialect Detection in Arabic Broadcast Speech (Ali et al.) [8] ▪ 5.4 Hyperparameter Tuning ▪ 5.5 Model Evaluation ▪ 5.6 Computational Resources
Lamya Alnahdi	▪ 1.1 Problem and Importance ▪ 1.2 Intended Task ▪ 2.2 Related Work: MGB-3: Arabic Speech Recognition Challenge Using YouTube Data [10] ▪ 7. conclusion
Maha Alruwais	▪ 1.2 Motivation for Choosing this Problem ▪ 1.3 Intended Task ▪ 2.2 Related Work: The Social Media Arabic Dialect Corpus (SMADC) (Alshutayri & Atwell) [3] ▪ 4. Methods
Norah Almadhi	▪ 3.1 Dataset: Casablanca [9] ▪ 2.2 Related Work: The Saudi Audio Dataset for Arabic Speech Recognition (SADA) [11] ▪ 6.1 Overall Performance Comparison ▪ 6.2 Generalization to Unseen Data ▪ 6.3 Insights and Observations
Maha Albakr	▪ 3.1 Dataset: SADA2022 [12] ▪ 5.1 Training Procedure, Feature Engineering, and Dataset Splitting ▪ 5.2 Data Augmentation and Regularization ▪ 5.3 Model Architecture Overview

Table 9: Specific Contributions of Team Members in Writing the Proposal

9. References

- [1] Y. Al-Tamimi, "Arabic Dialectology: Methods and Approaches," De Gruyter, 2017. [Online]. Available: https://api.pageplace.de/preview/DT0400.9783447199025_A38423520/preview-9783447199025_A38423520.pdf.
- [2] C. Holes, " Gulf Arabic," Taylor & Francis Group, 2003. [Online]. Available: <https://www.taylorfrancis.com/books/mono/10.4324/9780203218426/gulf-arabic-clive-holes>.
- [3] E. A. Areej Alshutayri, "Classifying Arabic dialect text in the Social Media Arabic Dialect Corpus (SMADC)," 2019. [Online]. Available: <https://aclanthology.org/W19-5607.pdf>. [Accessed November 2025].
- [4] B. McFee, "Librosa Feature Extraction," Librosa, 2024. [Online]. Available: <https://librosa.org/doc/latest/feature.html>.
- [5] O. Zaidan and C. Callison-Burch, "Arabic Dialect Identification," Association for Computational Linguistics, 2014. [Online]. Available: <https://aclanthology.org/P14-2062.pdf>.
- [6] B. McFee, "Librosa: Python Library for Audio and Music Analysis," GitHub, 2024. [Online]. Available: <https://github.com/librosa/librosa>.
- [7] S.-l. Org., "3.4. Metrics and scoring: quantifying the quality of predictions," Scikit-learn Org., [Online]. Available: https://scikit-learn.org/stable/modules/model_evaluation.html#classification-metrics. [Accessed November 2025].
- [8] A. Ali, P. Cardinal, S. Vogel and A. W. Black, "Automatic Dialect Detection in Arabic Broadcast Speech," *Interspeech*, p. 78–82, 2015.
- [9] UBC-NLP, "Casablanca: Arabic Dialect Dataset," Hugging Face, 2022. [Online]. Available: <https://huggingface.co/datasets/UBC-NLP/Casablanca>. [Accessed 09 2025].
- [10] P. B. J. G. e. a. A. Ali, "MGB-3: Arabic speech recognition challenge," in *2017 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, Okinawa, 2017.

- [11] M. P. S. B. B. Gerazov, "Arabic ASR on the SADA large-scale Arabic speech corpus with transformer-based models," arXiv, 2025.
- [12] D. Cai, "SADA2022: Saudi Arabic Dialect Audio Dataset," SDAIA & SBA, 2022. [Online]. Available: <https://www.kaggle.com/datasets/sdaiancai/sada2022>. [Accessed 09 2025].
- [13] T. Sainburg, "Noise reduction in python using spectral gating," PyPi, [Online]. Available: <https://pypi.org/project/noisereduce/>. [Accessed 11 2025].
- [14] Google, "Colaboratory," Google, [Online]. Available: <https://research.google.com/colaboratory/faq.html>. [Accessed November 2025].
- [15] G. Cloud, "Machine families resource and comparison guide," Google Cloud, [Online]. Available: <https://docs.cloud.google.com/compute/docs/machine-resource>. [Accessed November 2025].
- [16] "External data: Local Files, Drive, Sheets, and Cloud Storage," Google Colab, [Online]. Available: <https://colab.research.google.com/notebooks/io.ipynb>. [Accessed November 2025].
- [17] P. S. S. (. Ecosystem), "SciPy: Open source scientific tools for Python.," SciPy, [Online]. Available: during model training and evaluation. [Accessed November 2025].
- [18] C. Bisogni, "Acoustic features analysis for explainable machine learning," *Speech Communication*, vol. 157, p. 71–84, 2024.
- [19] Y. Lei and J. H. L. Hansen, "Dialect classification via text-independent training and adaptation," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 19, no. 1, p. 85–96, 2011.
- [20] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg and O. Nieto, "librosa: Audio and music signal analysis in Python," in *Proceedings of the 14th Python in Science Conference (SciPy)*, Austin, TX, 2015.
- [21] [Online].
- [22] A. E. E. M. B. N. M. Abdul-Mageed, "ARABERT and MARBERT: Deep bidirectional transformers for Arabic," 2020.
- [23] e. a. M. Abdul-Mageed, "Advances in Arabic dialect identification with large transformers," arXiv, 2024.

- [24] N. Kalach, "Gulf Arabic: Recent Data About the Variety Spoken in the Emirate of Dubai," OpenEdition Books, 2019. [Online]. Available: <https://books.openedition.org/iremam/pdf/3931>.